

Operating Systems

2520030273

K.Navadeep

SEC-5

WEEK-1

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using `fork()`. 3. Execute the command in the child process using an appropriate `exec()` system call. 4. Allow the parent process to wait for the child using `wait ()`. 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (`uname`, `lscpu`, `lsblk`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Code

```
kolasani_navadeep@Navadee × + v —
GNU nano 8.7.1 linux.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    char command[50];

    printf("Enter command: ");
    scanf("%s", command);

    pid_t pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
    }
    else if (pid == 0)
    {
        printf("Child PID: %d\n", getpid());

        execlp(command, command, NULL);

        printf("Command execution failed\n");
        exit(1);
    }
    else
    {
        printf("Parent PID: %d\n", getpid());
        wait(NULL);
        printf("Child completed\n");
    }

    return 0;
}
```

output

```
kolasani_navadeep@Navadeep-Vitus:~$ nano linux.c
kolasani_navadeep@Navadeep-Vitus:~$ gcc linux.c
kolasani_navadeep@Navadeep-Vitus:~$ ./a.out
Enter command: ls
Parent PID: 535
Child PID: 536
Current.c  Null.c      a.out  fork.      linux.c    week1.c.save
Downloads  Processid.c  f1     fork.c     p1.c       who
FCFS.c     RoundRobin.c f2     hi         process.c
FCFS.coxx  SJF.c        f3     hiiiiiiiiii week1.c
Child completed
```